

Cortex-M85에서의 draft-10 X-Wing 종단간 최적화: 동일 ELF 누적 측정과 함수 스티칭의 한계

익명 저자

익명 기관

anonymous@example.com

Abstract. 본 논문은 Arm Cortex-M85에서 draft-10 X-Wing 하이브리드 KEM을 종단간 최적화하고, 채택된 모든 변경의 누적 효과를 하나의 ELF에서 직접 측정한다. 최종 스택은 ML-KEM의 forward/inverse NTT와 current-order store, MVE packing/CBD/message 변환, X25519 fixed-base comb, X25519 곱셈과 제곱의 재스케줄, 그리고 정렬 인식 워드 단위 `memcpy/memset`으로 구성된다. draft-10 프로토콜과 입력을 양쪽에 고정된 A-B-B-A 실험을 독립 플래시 두 번, 셀당 100회 수행하고 네 paired 효과 중 최솟값을 취한 결과, 기준형 대비 키생성 **25.967%**, 캡슐화 **20.249%**, 확장 키 warm 역캡슐화 **11.224%**, packed-key cold 역캡슐화 **18.298%**의 사이클 감소를 얻었다. 디스패처를 제거한 최종형의 절대 성능은 각각 627,069, 988,456, 780,020, 1,406,887 사이클이다. 모든 KAT, 8-seed A/B 전 출력, 암묵적 거부, stack 게이트가 통과했고 독립 실행 간 효과 차이는 최대 0.00116%p였다. 한편 함수 스티칭(function stitching)은 국소적으로는 성립했다. 스칼라 명령과 MVE 명령의 동시 발행을 실리콘에서 100% 확인하고 제약 해결기 기반 2-스트림 스케줄러도 구축했지만, 실제 X25519 제곱과 NTT를 합친 커널은 순차 실행보다 느렸다. 이 양성·음성 결과를 함께 보면, 순차 실행 코어에서는 빈 발행 슬롯의 존재만으로 종단간 이득을 예측할 수 없으며 레지스터 수명, 재물질화, 저장 차단, API 경계를 포함한 동일 ELF 측정이 최종 판정에 필수적이다.

Keywords: PQ/T 하이브리드 · X-Wing · ML-KEM · X25519 · Cortex-M85 · MVE · 동일 ELF 측정 · 함수 스티칭 · SLOTHY

1 서론

양자 능력을 갖춘 공격자에 대한 과도기적 해답은 하이브리드 키 확립이다. draft-10 X-Wing [5]은 X25519와 ML-KEM-768을 함께 실행하고 두 비밀을 SHA3-256으로 결합한다. 둘 중 한 구성요소가 깨지더라도 다른 구성요소가 안전하면 결합 비밀은 보호되지만, 구현은 두 알고리즘의 비용을 모두 부담한다. 데스크톱급 코어에서는 이 하이브리드 세금이 작을 수 있지만, 임베디드 TLS 스택을 떠받치는 마이크로컨트롤러에서는 배포 여부를 좌우한다. 기존 완화책은 각 프리미티브를 따로 빠르게 만들거나 실리콘을 더 쓰는 것이다—두 번째 코어를 붙이는 식이다 [12,6]. 우리는 단일 Cortex-M85에서 ML-KEM, X25519, 메모리 접촉 코드, 그리고 두 알고리즘 사이의 스케줄까지 한 실험 계보 안에서 최적화하고, 마지막에 모든 채택 변경을 같은 ELF에서 함께 판정한다.

기회는 마이크로아키텍처에 있다. Cortex-M85는 순차 실행 듀얼이슈 Armv8.1-M 코어다. X25519의 체(field) 연산은 `umaal` 곱셈-누산과 `adds/adcs` 캐리로 이어

지는 긴 의존 사슬이다. 측정 결과 IPC는 1.03, 즉 2-이슈 용량의 48.7%가 높고 있다(절 3). ML-KEM의 대칭 연산 비용을 지배하는 Keccak-f1600 순열은 조밀하고 서로 독립적인 비트 연산(IPC 1.66)이며, 곱셈기가 아니라 ALU/시프트 자원을 사용한다. 두 계산은 Intel의 함수 스티칭 백서 [7]가 AES×SHA-1에 대해 활용했던 바로 그 의미에서 상보적이다—다만 그 작업은 비순차 x86을 대상으로 했고, 명령어를 소스 수준에서 손으로 배치했으며, 자신이 채운다고 주장한 유휴 용량을 한 번도 측정하지 않았고, 대칭 프리미티브끼리만 짝지었다.

구체적인 기여는 다음과 같다.

1. **draft-10 준거 최종 스택의 동일 ELF 누적 실측 (절 7).** 기준형과 최종형이 같은 프로토콜, 입력, 측정 함수, 메모리 배치를 공유하게 하고 일곱 전환 축을 한 번에 토글했다. 보수적인 최종 감소율은 키생성 25.967%, 캡슐화 20.249%, warm 역캡슐화 11.224%, cold 역캡슐화 18.298%다. 따라서 모든 경로가 빨라졌고 키생성과 캡슐화는 20% 목표를 넘었다.
2. **프로토콜 버전과 API 프로파일의 분리.** 2024/039 정의 [3]의 legacy wrapper를 draft-10의 combiner 순서, 32-byte packed secret key, encapsulation-key 검사로 이동했다. warm과 cold 역캡슐화를 별도 위크로드로 보고하고, 디스패치가 없는 최종형의 절대 cycle을 따로 동결했다.
3. **자동화: SLOTHY의 2-스트림 확장 (절 4).** 제약 해결 초최적화기 [1]에 빠져 있던 스칼라 명령어 클래스, 솔버 의존성으로서의 APSR(캐리 플래그) 수명, 스칼라 듀얼이슈 실행 모델, MAC 누산기 포워딩 규칙을 추가하고—17, 156, 1,184 명령어 규모에서 실리콘 검증했다. 독립적인 두 스트림을 단순히 이어 붙인 것 외에 아무 힌트도 주지 않았는데도, 솔버는 인터리빙을 스스로 찾아내고 스트림 사이로 레지스터를 재할당하여 모든 규모에서 최고의 수작업 스티치를 이기고, 손으로 최적화된 부품들로 조립한 강한 직렬 기준선까지 이긴다.
4. **매크로 이득이 실제 통합에서 사라지는 이유에 대한 정량적 해명 (절 5).** 사전 등록된 연산당 상한은 21–26%이고 선택 U의 매크로 커널은 7.82–10.32%(평균 8.89%)를 투영했다. 그러나 같은 Fiat 구현끼리 비교한 실제 U는—처음에는 3.99–19.81% 느렸고, 캡슐화의 큰 손실이 U8 코드의 플래시 배치 오류였음을 단일 변수 통제로 밝혀 ITCM으로 교정한 뒤에도—여전히 1.88–3.72% 느렸다. 우리는 이 반전을 분리 실험과 직접 통합으로 추적한다: 스피ل 자기잠식(완전 비율 커널은 3.3%만 낸다), 구조적 레지스터 양보 세금(레지스터를 아무리 적게 내놓아도 +12–14%), LSU 경합(순수 MAC 사슬은 58% 은닉, 스피ل 트래픽이 있으면 27%), 곱셈기 독점(스칼라×스칼라 NTT형 짝짓기는 −1% 은닉). 이어지는 짝짓기 스펙트럼 실험은 `vstrw`가 스칼라 발행을 막는다는 사실을 확인한다. 다만 실제 혼합 커널에서의 단일 변수 대조는 노출 223 사이클 중 110 사이클만 이 명령어에 귀속하며, 나머지 113 사이클은 미구명으로 남긴다. 마지막으로 실제 ladder 제어, 상태 유지와 코루틴/API 경계의 비용을 포함한 중단간 대조가 매크로 커널 투영의 충실도 한계를 확정한다.
5. **4-way MVE 배치 Keccak-f1600과 MVE packing/CBD (절 6):** 벡터 레지스터에 네 개의 순열 상태를 얹어, 상태·라운드당 188.5 사이클로 pqm4 스칼라 기준 230.3 사이클 대비 −18%를 스칼라 데이터 레지스터 0개로 달성한다. 또한 ML-KEM의 packing/압축/CBD/메시지 변환 11개 함수의 MVE 구현(C9)은 실제 X-Wing 중단간에서 1.46–4.66%를 추가로 절감하며, 상수 시간 성질을 보존한다(고정 루프와 술어 실행만 사용). 두 결과 모두 모든 Armv8.1-M ML-KEM에 독립적으로 유용하다.

6. 사전 등록 방법론 (절 3). 이 프로젝트의 모든 진행/중단 판정은 측정 전에 문서화되었다: 상한 측정 직후의 킬 게이트, 중단간 단계의 20% 기준선, 그리고 명시된 대체 서사. 우리는 그 약속들에 비추어 결과를 보고하며, 약속을 지키지 못한 부분도 함께 보고한다.

이 논문은 성공한 최적화만 나열하지 않는다. 채택된 일곱 측이 만든 중단간 이득과, 성공하지 못한 스티칭 계열에서 드러난 병목을 같은 측정 규율로 보고한다. 이 구분이 있어야 국소 커널의 개선률을 중단간 결과처럼 더하는 오류를 피할 수 있다.

2 배경

2.1 함수 스티칭과 그 약속

Intel 백서 [7]는 독립적인 두 계산을 엮어 한쪽의 스톱을 다른 쪽이 매우게 하며, 이상적으로는 $T_{\text{stitch}} \approx \max(T_A, T_B)$, 즉 짧은 쪽 계산이 공짜가 된다. 그 보고서는 소스 수준의 수작업 스케줄이고, 채워지는 유휴 용량에 대한 측정을 전혀 제시하지 않으며, 스케줄링의 불완전함을 흡수해 주는 비순차 하드웨어에 의존한다. 우리가 그대로 채택하는 원칙이 하나 있다: 이득은 사용 가능한 가장 강한 직렬 기준선—작업 도중 발견한 기준선 자체의 최적화까지 포함하여—에 대해 보고해야 한다.

2.2 Cortex-M85: 천장의 산술

순차 실행 듀얼이슈 코어에서 슬롯 회수의 상한은 50%다(Intel의 4-이슈 Westmere는 최대 75%까지 낭비했다). 하드웨어 재정렬이 없으므로 정적 스케줄의 품질이 곧 성능이다. 이는 양날이다: 스티칭이 효과를 보려면 완벽하게 스케줄링되어야 하고, 따라서 수작업 배치가 아니라 제약 해결기가 자연스러운 도구가 된다. 우리는 메모리 계층에 의한 변동을 제거하기 위해 코드를 ITCM에, 데이터를 DTCM에(둘 다 0-wait) 고정한다.

2.3 X-Wing과 그 안의 Keccak 지분

draft-10 X-Wing [5] = X25519 + ML-KEM-768 + SHA3-256 결합기다. 키 생성은 X25519 스칼라 곱셈을 한 번, 캡슐화는 두 번, 역캡슐화는 한 번 수행하며, ML-KEM 쪽은 Keccak-f1600을 각각 43, 44, 44회 호출한다(실측). 여기에 결합기 순열 하나가 더해진다. 따라서 자연스러운 스트림 쌍은 X25519(MAC 바운드 스트림 A) × Keccak(ALU 바운드 스트림 B)이다.

2.4 SLOTHY

SLOTHY [1,2]는 명령어 스케줄링과 레지스터 할당을 CP-SAT로 동시에 푼다—단, 단일 입력 스트림에 대해서다. 이 도구의 Armv8.1-M 모델은 MVE 중심이어서 X25519가 필요로 하는 스칼라 클래스(`umull/umaal`, 캐리 산술)가 없었고, 스칼라 듀얼이슈가 모델링되지 않았으며(발행물 1), 플래그 수명도 제약이 아니었다. 바로 이것들이 절 4의 확장 항목이다.

Table 1. 단독 실행 측정치와 슬롯 낭비(사이클 중앙값).

커널	사이클 명령어 수 IPC 슬롯 낭비			
X25519 스칼라 곱셈 (Lenngren)	357,474	366,934	1.03	48.7%
Keccak-f1600 (pqm4)	5,526	9,162	1.66	17.1%
ML-KEM-768 키생성/캡슐화/역캡슐화	437k/453k/504k	(순열 43/44/44회)		
SHA3-256 결합기 (134 B)	7,648	—	—	—

3 사전 등록된 상한: 얻을 것이 얼마나 있는가?

방법. 모든 측정 조건: EK-RA8M1(Cortex-M85, 480 MHz), GCC 13.2.1 -O2, 코드는 ITCM, 데이터/스택은 DTCM, DWT CYCCNT, $N=100$ 의 중앙값, 25 사이클의 하네스 보정치를 차감. 명령어 수는 Unicorn 에뮬레이션에서 얻었고 서로 무관한 두 입력에서 교차 확인했다(두 입력의 명령어 수가 같다는 것 자체가 상수 시간 점검을 겸한다, [절 8](#)). 에뮬레이션은 오직 분자에만 쓰인다. 두 커널 모두 비밀 독립적인 직선 코드이므로 명령어 수는 추정치가 아니라 결정적 상수이며, 모든 사이클 수치는 실리콘에서 측정한 값이다. 남은 불확실성은 정의상의 문제다—에뮬레이터가 IT 블록 본문을 개별적으로 계산하므로 INST_RETIRED와 몇 퍼센트 차이가 날 수 있다—따라서 슬롯 낭비 수치에는 대략 $\pm 1\%$ 의 오차가 붙는다. 기능 게이트: RFC 7748 KAT [\[9\]](#), SHA3-256 KAT, pqm4 [\[8\]](#) 위의 ML-KEM-768 왕복 [\[11\]](#), 그리고 Lenngren X25519 [\[10\]](#)—모두 보드에서 통과했다. 측정에 앞서 우리는 판정 규칙을 등록했다: 상한 $<15\%$ 면 프로젝트 중단, $15\% \leq b < 30\%$ 면 축소된 2주 파일럿, $\geq 30\%$ 면 진행.

[표 1](#)은 Intel이 한 번도 측정하지 않았던 것, 즉 유틸 용량을 정량화한다. X25519의 의존 사슬은 스칼라 곱셈 한 번당 $2 \cdot 357,474 - 366,934 = 348,014$ 개의 빈 발행 슬롯을 남긴다. Keccak의 명령어를 그 슬롯에 대고 (스트림 길이로 상한을 씌워) 계산하면 [표 2](#)이 나온다: X-Wing 연산당 21–26%, 평균 23.9%다. 슬롯 항은 모든 유틸 슬롯이 파트너 스트림으로 채워질 수 있다고 가정하는데—순차 실행 코어에서는 낙관적인 가정이지만—[절 4](#)에서 1:1 인터리브가 IPC 1.99, 즉 2-이슈 하한에서 0.5% 이내를 기록하므로 사후적으로 정당화된다. 구조적으로 두 가지가 눈에 띈다: (i) 캡슐화는 스트림 길이에 의해 제한된다—스칼라 곱셈 두 번이 만드는 유틸 슬롯이 Keccak의 명령어 수보다 많다. (ii) M85에서는 Keccak 스트림이 짧은 쪽이므로, 상한을 정하는 것은 슬롯 용량이 아니라 커버리지 희석이다. 이는 우리의 사전 등록이 예상했던 바다. 계획서는 M4 기준 회계 [\[13\]](#)로부터 X25519가 더 긴 스트림이 되고 커버리지가 희석될 것이라고 예측했으며, M85 측정은 그 방향을 확인해 준다. 게이트 판정을 기계적으로 적용하면: $23.9\% \in [15\%, 30\%) \Rightarrow$ 파일럿.

분모, 그리고 사전 등록으로부터의 이탈 한 건. 본 논문의 모든 종단간 백분율은 분모가 하나다: **X-Wing 연산 1회를 순차 하이브리드로 끝까지 실행한 전체 사이클**. 여기에는 스티칭의 대상이 아닌 ML-KEM의 비-Keccak 구간(NTT, CBD, 인코딩, 접착 코드)이 포함되며, 그 구간은 연산에 따라 전체의 17.8–30.0%를 차지한다. 우리의 사전 등록 문서는 상한을 $\text{overlap}/(\text{cyc}_A + \text{cyc}_B)$ 로 적었고 스트림 B 를 “ML-KEM 내부의 Keccak”으로 정의했다. 그 두 문장을 문자 그대로 결합하면 분모에서 비-Keccak 구간이 빠지고 상한은 25.8–35.3%, 평균 31.9%가 되어 사전

Table 2. X-Wing 연산별 사전 등록된 중첩 상한. 분모는 X-Wing 연산 1회의 전체 사이클 (794,198 / 1,175,479 / 868,772)이며 ML-KEM의 비-Keccak 구간을 포함한다.

연산	전체 사이클	중첩 가능 사이클	상한
키 생성	794,198	209,905	26.4%
캡슐화	1,175,479	248,670	21.2%
역캡슐화	868,772	209,905	24.2%

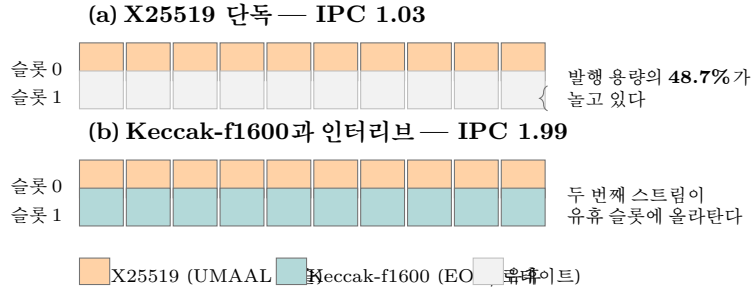


Fig. 1. 순차 실행 듀얼이슈 코어에서 스티칭이 이득이 되어야 하는 이유. 몽고메리 사다리 한 단계의 곱셈-누산 사슬은 자기 자신의 캐리 의존성 때문에 직렬화되고, 그 결과 전체 사이클의 48.7%에서 두 발행 슬롯 중 하나가 비어 있다(실측, 표 1). 레지스터가 겹치지 않는 두 번째 명령어 스트림은 첫 번째를 밀어내지 않고 그 슬롯을 차지할 수 있으며, 절 4의 마이크로벤치마크는 2-이슈 하한인 IPC 1.99에 도달한다. 이 그림과 실제로 U 매크로커널 투영 7.82–10.32%와 실제 직접 통합의 음의 결과 사이의 간극이 본 논문의 주제다.

등록된 결정표의 다른 칸 (“직진”, $\geq 30\%$)에 떨어진다. 우리는 더 넓은—그리고 우리에게 불리한—분모를 채택했고, 그 결과가 “파일럿” 판정이다. 이것을 이탈로 신고한다. 근거는 중단간 이득을 주장하는 논문의 분모는 중단간 연산이어야 한다는 것이다. 좁은 분모는 스티칭이 도달할 수 없는 작업을 감추어 이득을 부풀린다. 이 선택은 게이트 판정 시점부터 계산에 반영돼 있었으나 명시되지는 않았으며, 사후에 명문화했다.

4 자동화: 2-스트림 SLOTHY

4.1 먼저 손으로 확인한 실현 가능성

마이크로벤치마크는 하드웨어가 인터리빙을 소화한다는 것을 보여 준다. UMAAL 사슬과 플래그를 세우지 않는 eor/로테이트 사슬을 1:1로 섞으면 IPC 1.99(2-이슈 하한에서 0.5% 이내)로 돌아가며, 짧은 쪽 스트림의 80%를 숨긴다. 두 번째 스트림을 메모리 상주로 바꾸어도 절감량은 4,000 사이클 중 2 사이클만 달라진다. LSU 트래픽이 MAC 스트림과 짝을 이루므로 스티칭 고유의 스펴 페널티는 0이다. 스트림 B를 위해 레지스터 4개(심지어 8개)를 예약해도 컴파일된 스트림 A에는 측정 가능한 비용이 발생하지 않는다—다만 이는 컴파일러에 아직 스펴 여유가 있는 경우에만 성립한다. 손으로 튜닝한 fiat 곱셈은 대신 12–14%의 양보 세금을 묻다(아래에서 정량화). 그래서 이 마이크로벤치마크 결과를 일반적인 결론으로

읽어서는 안 된다. 이어서 스크립트로 만든 “지퍼(zipper)” 스티치가 KAT로 검증된 실제 커널을 생산한다. 그 최고 수작업 결과인 256비트 곱셈 하나 + 완전한 Keccak 라운드 하나에 681 사이클은, 여전히 627 사이클짜리 최강 직렬 기준선의 1.09×이며 수작업 스왑은 거기서 정채되었다. 그 격차는 레지스터 할당과 스케줄링의 동시 최적화 문제—즉 솔버의 직무 기술서다.

4.2 솔버에 필요했던 것

우리 패치(공개; 상류 모델은 241093fa5ddc)는 발견 순서대로 다음을 기여하며, 각 항목은 실측된 실패가 강제된 것이다:

- **Armv8.1-M 모델에 없던 스칼라 명령어 클래스 19종** (umull, umaal, 즉시값 형태를 포함한 캐리 산술, umlal, 시프트/플래그 변종 등).
- **의존성으로서의 APSR 수명** (modifiesFlags/dependsOnFlags): X25519 캐리 사슬이 어떤 합법적 스케줄에서도 구성상 살아남게 하여, Intel의 대칭 전용 짜집기가 무시하고 넘어갈 수 있었던 유일한 해저드를 기계화한다.
- **스칼라 듀얼이슈**: 발행률 1 → 2 및 두 번째 스칼라 ALU 슬롯 추가 (MAC와 LSU는 그대로).
- **MAC 누산기 포워딩** (모델 v0.3): 실리콘은 의존적인 umaal 사슬을 누산기 오퍼랜드를 통해 명령어당 1 사이클로 실행한다. 기본값인 2 사이클 지연은 스트림 A의 가격을 2×로 잘못 매겨, 수정 전까지 솔버가 지퍼에게 졌다(1,319 대 1,018 사이클). 수정 후에는 이긴다(996). 모델 정확도는 부차적인 미덕이 아니라, 순차 실행 코어에서는 승패를 가르는 요소다.
- **컴파일러 출력을 솔버에 먹이기 위한 안전 변환 규칙 3가지**. 각각은 우리가 0까지 디버깅한 실제 손상 사례로 검증되었다: 더블워드 접근을 분할할 것(모델은 ldrd/strd를 4바이트로 취급한다), 스펠 슬롯을 절대 재사용하지 말 것 (WAR/WAW 메모리 반의존성이 보존되지 않는다), 베이스를 파괴하는 로드 쌍을 마지막에 배치할 것. 세 건의 이슈로 상류에 보고했다.

입력에는 아무 주석도 필요 없다. 두 스트림을 이어 붙이면(또는 아래처럼 미리 지퍼로 엮으면) 의존성 분석이 그들의 독립성을 스스로 발견한다.

4.3 검증 사다리

[표 3](#)은 이 도구의 성적표다. 세 가지를 짚는다. 첫째, 솔버는 수작업 스티치가 존재하는 모든 지점에서 그 최고 기록을 이기며, 완전 규모에서는 수작업 기법을 아예 구성할 수조차 없다. 지퍼의 전제—스트림 사이의 정적 레지스터 분할—은 스트림 A가 모든 레지스터를 자유롭게 쓰는 순간 무너지고(우리는 그 결과로 생기는 라이브 값 손상을 하드 폴트로 재현해 보았다), 스트림을 가로질러 전역적으로 할당하는 솔버에는 그런 전제가 없다. 둘째, 충실도 사다리는 실제 재료에서 끝난다. fiat-crypto 체 곱셈 (BoringSSL의 carry-mul)과 pqm4 표현(비트 인터리브) Keccak 라운드의 조합은 579 사이클에 도달한다—직렬 대비 9% 아래이고, 손으로 최적화된 부품들로 조립한 626 사이클 최강 직렬 기준선보다 7.5% 아래인데, 스트림 A가 컴파일러 출력임에도 그렇다. 수작업 시절의 격차는 자동화가 메웠다. 셋째, 모델 충실도: 예측 오차는 소규모에서 <1%, 완전 규모에서 5-7%(분할 원도 경계 스톱), v0.4 스토어 수정 이후 스칼라×MVE에서 12%([절 5](#)).

Table 3. 직렬 대 최고 수작업 스티치 대 솔버, 보드 실측 (반복당 사이클, C 레퍼런스 대비 정확성 검증 완료).

규모	직렬 수작업 SLOTHY			솔버 대 수작업
17 명령어 (개념 검증)	13.06	—	11.07	(직렬 대비 -15%)
156 명령어	110.06	88.07	85.06	-3.4%
1,184 명령어 (완전 커널, 역사 세대)	727.0	681	657.0	-3.5%
1,184 명령어 (완전 커널, 현재 동일 빌드)	727.0	662.4	654.0	-1.3%
1,130 명령어 (fiat carry-mul)	702.0	—	630.0	(직렬 대비 -10.3%)
1,110 명령어 (비트 인터리브)	636.0	—	579.0	(직렬 대비 -9.0%)
동시 발행 1,222 명령어 (스칼라×MVE)	1,365	1,013	996	-1.7%

Table 4. 상한과 실측 사이 격차의 원인들, 각각 실리콘에서 분리 측정.

원인	분리 실험	결과
(a) 스피ل 자기잠식	완전 비율 fiat 커널 (2,526 명령어)	상한 21%+ → 3.3%
(b) 레지스터 양보 세급	-ffixed 2개 대 3개	+12.1-13.5%, 개수 무관
(c) LSU 경합	실제 곱셈(스피ل 있음) 대 순수 MAC 사슬	은닉 27% 대 58%
(d) 곱셈기 독점	스칼라×스칼라 NTT형	은닉 -1%
(e) 파이프 분리 (대조군)	스칼라×MVE NTT형	은닉 62%

작업 단위 하나(곱셈 하나 + 라운드 하나)의 변천사가 이 프로젝트를 요약한다: 875 → 799 → 681 → 657 → 630 → **579** 사이클, 최초 지퍼 대비 -34%(같은 세대 안의 계보). 마지막으로 완전 커널의 재대결이 도구의 현재 상태를 확정한다. 커널 재생성 이후의 현재 빌드에서 같은 입력을 새로 풀자 솔버는 새 스케줄을 찾아 순차 727.0, 수작업 지퍼 662.4에 대해 654.0 사이클을 두 독립 실행에서 정확히 재현했다—수작업 대비 -1.3%로, 사전 등록한 성공 조건을 현재 세대에서도 충족한다.

여기서 도출된 실용 파이프라인은 이렇다: 실제 산술은 C로 작성하고, 스트림 B의 레지스터는 -ffixed로 예약하고(컴파일러가 올바른 스피를 삽입해 준다), 스케줄링은 솔버에 맡긴다. 14개 레지스터를 모두 포화시키는 순수 작성 어셈블리는 스피를 추가하지 않고서는 어떤 스케줄러로도 스티칭할 수 없는데, 스케줄러는 스피를 추가할 권한이 없다.

5 회수량이 얇은 이유: 분해

상한은 21-26%라고 말했고 U 선택 경로의 완전 비율 매크로커널은 X-Wing 전체 분모에서 7.82-10.32%를 투영했다. 그러나 실제 U 직접 통합은 세 연산 모두 음수였다 (절 6). 이 절이 본 논문의 핵심이다. 각 실험은 한 원인을 드러내도록 설계했으며, 끝내 분해되지 않은 잔차도 그대로 남긴다.

(a) 스칼라 계획은 스스로를 잡아먹는다. 실제 X-Wing 비율(라운드당 곱셈 네 번, 2,526 명령어)로 스티칭하면 1,573 → 1,521 사이클, 즉 **3.3%**로 상한보다 한 자릿수 아래다. 메커니즘은 이렇다. 스트림 B에 레지스터를 내주기 위해 (-ffixed로 컴파일된) 스트림 A가 스피하고, 그 스피 로드/스토어가 하필 스트림 B의 메모리

상주 상태가 필요로 하는 바로 그 LSU 슬롯을 차지한다. 유틸 용량은 실재하지만, 그것을 채우는 행위가 그것을 소비한다.

(b) 양보 세금은 구조적이다. 레지스터 3개를 내주면 fiat 곱셈이 +13.5%를 묻다 (287.6 → 326.5). 2개만 내주어도 +12.1%다. 이 근사적 동일성이 바로 발견이다. 세금이 사는 것은 두 번째 라이브 레지스터 집합의 존재 자체이며—컴파일러가 라이브 레인지를 재구성한다—개수에 대해서는 거의 평평하다. (절 4의 마이크로 벤치마크에서 예약이 공짜였던 것과 대조된다. 그 스트림에는 여유가 있었지만, 캐리로 포화된 실제 코드에는 없다.)

(c) 정량화된 LSU 경합. 순수 umaal 사슬은 MVE Keccak 스트림의 58%를 숨기고, 스피드 트래픽을 동반한 실제 곱셈은 27%를 숨긴다. 이 차이는 A의 스피드와 B의 상태 접근 사이의 로드/스토어 유닛 경쟁이다.

(d-e) 짝짓기 규칙. 스칼라×스칼라 NTT형 짝짓기(두 스트림 모두 곱셈기 바운드)는 -1%를 숨긴다. MAC 파이프는 독점 자원이며, 이 음성 대조군은 다른 곳에서 얻은 이득이 측정 기법의 산물이 아니라 상보성에서 왔음을 검증한다. 스트림 B를 MVE의 별도 곱셈기로 옮기면 같은 실험이 62% 은닉으로 뒤집힌다—혼합 레지스터 파일 쌍이 쉬운 경우라는 Intel의 정성적 주장을, 임베디드 순차 실행 환경에서 실측한 판본이다.

동시 발행 천장과 남은 잔차. 스칼라×MVE 계획에서 은닉률은 60% 부근에서 포화된다. 짝짓기 스펙트럼 실험(umaal 사슬에 대해 MVE 클래스를 하나씩 붙여 보는 실험)은 중요한 차단 클래스를 지목한다. veor, vldrw, vshl/vsri는 모두 99%로 짝을 이루고—vldrw는 2 사이클 명령어이면서도 두 사이클 내내 사이클당 스칼라 하나를 허용한다—반면 vstrw는 두 사이클 어느 쪽에서도 하나도 허용하지 않는다(-1% 은닉). 그러나 이 합성 단가를 실제 혼합 커널에 그대로 곱한 $121 \times 2 = 242$ 사이클 모델은 후속 단일 변수 실험 Q에서 반증됐다. 명령 수와 B의 단독 비용을 고정한 채 121개 vstrw만 값 보존 vorr로 바꾸자 노출은 $223 \rightarrow 113$ 사이클이 되었다. 따라서 실제 기여는 110 사이클(스토어당 0.91 사이클)이고, 잔여 113 사이클은 아직 귀속되지 않았다. 이 차단 규칙을 모델에 인코딩하자(스토어가 STORE+SCALAR를 점유; v0.4) 예측 오차는 27%에서 12%로 줄었다. 다만 정직하게 덧붙이면, v0.4로 재스케줄된 커널은 실리콘에서 v0.3 스케줄을 이기지 못했다(1,018 대 996). 853 사이클의 하위 천장(B 스톤 제거)은 이제 측정된 하한이 아니라 폐기된 모델의 추정치다.

6 플랜 B 결과: 스칼라 × MVE, X-Wing 투영

(a)-(c)를 감안하면 레지스터 파일이 분리된 계획이 배포 가능한 쪽이다. X25519는 14개 범용 레지스터를 모두 쓰는 스칼라로 남고, Keccak-f1600은 MVE로 옮긴다.

4-way 배치 MVE Keccak-f1600. Armv8.1-M MVE에는 128비트 벡터 레지스터가 여덟 개 있다—레인 위치당 정확히 네 개의 32비트 인터리브 Keccak 상태다. 우리 생성기는 레인 전반에 걸쳐 회전량을 균일화하고 상태 주소 지정을 고정하는 방식으로 순열을 배치하여, 베이스 레지스터 개수를 $5 \rightarrow 3 \rightarrow 2$ 로 비용 없이 줄인다

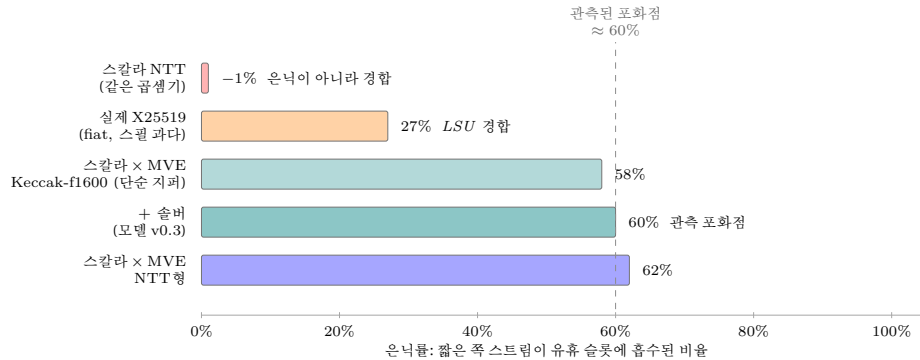


Fig. 2. 짝짓기와 파이프에 따른 은닉률. 실행 자원을 공유하는 두 스트림은 아예 스티칭되지 않는다(UMAAL 사슬에 맞선 스칼라 NTT: -1%, 곱셈기를 둘러싼 줄다리기). 실제의 스필 과다 X25519는 그 스필 트래픽이 로드/스토어 유닛에서 파트너와 경합하기 때문에 짝이 잘 맞지 않는다(27%). 파트너를 다른 파이프(MVE)로 옮기면 슬롯의 대부분을 되찾는다(58-62%). 벡터 스토어가 스칼라 발행 슬롯을 점유하는 것은 확인됐지만, 실제 혼합 커널의 노출 223 사이클 중 직접 귀속된 몫은 110 사이클이고 나머지 113 사이클은 미구명이다(절 5).

(vldrw의 ± 508 오프셋 활용). 결과는 **상대·라운드당 188.5 사이클로 pqm4 스칼라의 229 사이클 대비 -18%**이며, 스칼라 데이터 레지스터는 0개다—범용 레지스터 파일 전체가 X25519에 남는다. 레인 전치(transposition) 접착 코드는 2.5%를 쓴다(실측). ML-KEM-768의 연산당 43-44회 순열은 4-way 큐를 자연스럽게 채우고, X-Wing 결합기의 SHA3-256 순열 하나는 45번째 항목으로 같은 큐에 올라간다. 별도의 기전이 필요 없고 비용은 배치 라운드 하나 이하이다.

매크로커널 투영. 우리는 참 연산조합(fiat 곱셈: 배치 라운드 = 캡슐화 8:1, 키생성/역캡슐화 4:1; 실제 명령어 스트림, 플래시 상주, 정확성 검증 완료)에서 매크로 단위를 측정했다. 실험 U는 범용 레지스터 양보를 하나로 줄인 `yield1+b1`을 선택했다. 이 단위의 절감량과 호출 수를 X-Wing 전체 분모에 넣으면 keygen 8.54%, encaps 10.32%, decaps 7.82%(평균 8.89%)를 투영한다. 독립 MVE Keccak까지 산술적으로 더한 평균 13.65% 역시 당시의 미통합 공학적 투영이다. 아래 직접 통합 결과가 있으므로 두 수치는 최종 성능 주장이 아니라 투영 충실도를 진단하는 과거 예측값으로만 사용한다.

선택 U의 실제 직접 통합. AC가 b0였다는 코드 감사 뒤, 선택 U의 r11 고정·하위 연속 b1을 Fiat 10×25/26 X25519 Montgomery ladder의 임의 피연산자 경로에 새로 넣었다. 주 비교 F→U는 같은 Fiat X25519와 같은 x4 Keccak을 사용하므로 스티칭 자체의 차분이다. 첫 측정은 keygen -4.14%, encaps -19.78-19.81%, decaps -3.99%였다. 캡슐화 손실이 키생성의 8배라는 불균형을 추적한 사후 감사는 U 어셈블리(13.3 KiB의 U8 명령 워킹셋)가 핫 커널과 달리 일반 코드 플래시에 놓여 있었음을 발견했다. 명령열을 바꾸지 않고 배치만 바꾼 단일 변수 통제에서 동일 작업의 2-job 고유 손실 $P8 - 2P4$ 는 플래시 114.3-114.8k에서 ITCM 약 6k 사이클로 95% 줄었고, U8만 ITCM으로 옮긴 추가 통제에서는 음수가 되어 U8

Table 5. 매크로 투영과 실제 종단간 통합의 비교. 행마다 기준선이 다르므로 행 사이의 크기를 직접 비교하지 않는다. F는 후보와 같은 Fiat X25519 및 x4 경로의 순차 기준선이고, X/Y/C8은 실제 ML-KEM 최적화의 누적 통합 경로다.

결과	키 생성	캡슐화 역캡슐화 판정
U 매크로 투영	+8.54%	+10.32% +7.82% 과거 투영
실제 U, 플래시 배치 오류 포함	-4.14%	-19.78/-19.81% -3.99% 진단값
실제 U, ITCM 교정 후 (F 대비)	-1.98%	-3.72% -1.88% 기각
실제 X/Y/C8 (NTT 경로)	+2.85%	+1.54% +3.25% 채택 (구성)
실제 C9 (packing/CBD MVE)	+1.46%	+2.67% +4.66% 채택 (구성)
X/Y/C8 + C9, 동일 펌웨어 2×2	+4.29%	+4.16% +7.71% 중간 채택

의 구조적 추가 비용 가설이 기각되었다. ITCM으로 교정한 대표 결과는 독립 ABBA($N = 100$) 두 run에서 keygen -26,291/-26,362 사이클(-1.98%), encaps -83,434/-83,462 사이클(-3.72%), decaps -26,298/-26,302 사이클(-1.88%)이다. 음수 절감은 악화다. RFC 7748 KAT 네 개, SHA3 KAT, x4/U primitive KAT, 8-seed X-Wing full-output/reject, timing, stack/canary가 모두 통과했고 두 run 모두 harness_fails=0이었다. 별도의 체크포인트 계측은 남은 캡슐화 손실이 잔여 사다리의 직렬화가 아니라, 두 번째 job을 더 비싼 U8 융합 구간으로 옮긴 순효과(약 +55.6k 사이클)임을 귀속했다.

최초 파일럿에서는 스티칭 뒤 잔여 ladder 곱셈에도 후보에만 queue search/modulo dispatcher가 남아 있었다. 이를 판정에서 제외하고 측정 전 보정 기록 뒤 direct finish로 제거했다. 그 결과 파일럿의 손실은 크게 줄었지만 최종 부호는 세 연산 모두 음수였다. 실제 ladder 제어, 상태 전이와 코루틴/API 경계를 보존하는 비용이 매크로 커널에서 숨긴 슬롯보다 컸으며, X25519 job이 돌인 encaps에서 손실이 가장 컸다.

대조군과 채택 경로. 굵은 함수 단위 교대는 3:3 단위에서 아무것도 회수하지 못했지만(-0.7%), 같은 재료의 명령어 수준 스티치는 -8.6%였다. 따라서 국소 교차 발행 자체는 실재한다. 다만 이 국소 결과를 종단간 결과로 외삽할 수는 없다. 별도로 실제 ML-KEM에 순차 통합한 X/Y/C8(NTT 경로)은 keygen 2.85%, encaps 1.54%, decaps 3.25%를, packing/CBD/메시지 변환의 MVE 구현(C9)은 그 위에서 1.46-4.66%를 절감했고 각각 정확성 게이트를 통과했다. 두 경로를 같은 펌웨어에서 2×2 완전요인으로 측정하면(8셀 대칭 순서, 두 독립 실행) 누적 절감은 키생성 4.29%, 캡슐화 4.16%, 역캡슐화 7.71%이고 요인 간 상호작용은 -249+170 사이클로 0에 수렴한다—두 최적화는 독립적으로 합산된다. 이는 최종 스택에 들어간 중간 채택 결과이며, [절 7](#)의 누적 실험이 이를 대체한다. 후속 음성 결과 둘도 기록한다: C9의 바이트 끼워넣기까지 전부 gather/scatter로 벡터화한 변형은 오히려 1.0-1.3k 사이클 느려 기각했고(벡터 로드/스토어의 발행 비용이 스칼라 끼워넣기를 이긴다), MVE C9 코드의 ITCM 재배치는 $\pm 0.5k$ 사이클 이내로 효과가 없었다—U8의 114k대 플래시 손실과 달리 C9의 작은 워킹셋은 명령 캐시로 충분하다. AC의 b0+x4 복합 후보도 정확성은 통과했지만 X/Y/C8보다 세 연산 모두 느려 기각했으며, 선택 U의 직접 측정과 구분한다.

Table 6. 동일 ELF 누적 실험의 일곱 전환 축.

축	A: 기준형	B: 최종형
X/Y/C8	기존 NTT/invNTT와 store 순서	M85 NTT/invNTT, current-order store fusion
C9	scalar packing/CBD/message	MVE packing/CBD/message 변환
X06	basepoint 9에도 일반 ladder	precomputed fixed-base comb
X01	원본 <code>fe25519_mul</code>	평탄화·재스케줄한 곱셈
X02	원본 <code>fe25519_sqr</code>	명령 수 불변 재스케줄 제공
K31-copy	newlib-nano <code>memcpy</code>	정렬 인식 워드 복사
K31-zero	newlib-nano <code>memset</code>	정렬 인식 워드 영접화

스티칭 계열의 판정. 선택 U 자체는 같은 Fiat 기준에서 성능 실패다. 사전 등록된 대체안에 따라 2-스트림 자동화 도구, 원인 분해, 투영 충실도 반전의 직접 측정은 음성 결과로 보존한다. 그러나 프로젝트 전체의 20% 목표 판정은 이 중간 단계가 아니라, 모든 채택 축을 draft-10에서 함께 컨 [절 7](#)의 실험으로 내린다.

7 draft-10 최종 누적 결과

7.1 준거 이동과 API 프로파일

초기 wrapper는 2024/039 논문 [3]의 정의를 구현했다. 이후 IETF draft-10 [5]으로 준거를 옮기면서 세 경계를 동시에 바꿨다. 첫째, combiner 입력은 $ss_M \parallel ss_X \parallel ct_X \parallel pk_X \parallel \text{XWingLabel}$ 순서다. 둘째, decapsulation key는 32-byte seed를 SHAKE256으로 96바이트 확장해 ML-KEM과 X25519 재료를 만든다. 셋째, 캡슐화 전에 FIPS 203의 ML-KEM encapsulation-key 검사를 수행한다. 이 이동은 부분 적용할 수 없다. 일부만 바꾸면 어느 정의에도 속하지 않는 제3의 프로토콜이 된다.

역캡슐화는 두 프로파일로 분리했다. *warm*은 미리 확장한 비밀키 객체를 재사용하고, *cold*는 매 호출마다 packed 32-byte seed에서 키를 확장한다. 두 수치는 다른 API 워크로드이므로 하나의 “decapsulation latency”로 평균하지 않는다. draft-10 공식 test vector 1에 대해 shared secret 32바이트와 되짚기 decapsulation은 완전 일치했고, 확보한 public key 앞 128바이트와 ciphertext 앞 62바이트도 일치했다. 공식 문서에서 확보하지 못한 public key와 ciphertext의 나머지 바이트에 대한 외부 oracle 대조는 수행하지 못했으며, 이 한계를 결과에 포함한다.

7.2 한 번에 토글한 채택 스택

[표 6](#)의 일곱 전환 축을 하나의 `d10_set_optimizations(mode)`에 묶었다. A와 B는 프로토콜, 난수 스트림, 측정 함수, 스택, 코드 배치를 공유하고 오직 이 모드만 다르다. 각 축의 과거 개별 개선물은 서로 다른 분모에서 얻었으므로 여기서 더하지 않는다.

Table 7. draft-10 기준형 A와 최종형 B의 동일 ELF 누적 결과. 범위는 독립 두 run의 A-B-B-A 네 위치에서 관측한 중앙값이며, 마지막 열은 네 paired 효과의 최솟값이다.

연산	A 범위 (cycle)	B 범위 (cycle)	보수적 감소율
키생성	870,398–870,412	644,346–644,390	25.967%
캡슐화	1,321,225–1,321,235	1,053,684–1,053,691	20.249%
역캡슐화 (warm)	934,141–934,160	829,297–829,306	11.224%
역캡슐화 (cold)	1,801,440–1,801,455	1,471,808–1,471,818	18.298%

7.3 동일 ELF A-B-B-A 측정

EK-RA8M1의 Cortex-M85를 480 MHz로 고정하고 DWT CYCCNT로 각 셀을 100 회 측정해 중앙값을 취했다. 실행 순서는 A-B-B-A이며, 같은 ELF를 독립적으로 두 번 플래시해 총 네 paired 효과를 얻었다. 각 pair의 사이클 감소율은

$$g(A, B) = 100 \frac{A - B}{A} \quad (1)$$

으로 계산하고, 최종값은 네 g 중 최솟값으로 정했다. 이는 평균보다 불리하지만 run 선택에 따른 낙관을 막는다.

네 경로 모두 양의 효과다. 키생성과 캡슐화는 사전 등록한 20% 목표를 넘었고, warm과 cold 역캡슐화는 개선됐지만 20%에는 못 미쳤다. 연산마다 호출 구성과 분모가 다르므로 네 백분율의 단순 평균을 “전체 개선률”로 보고하지 않는다.

정확성과 재현성. 두 run 모두 RFC 7748, FIPS 203, SHA3/SHAKE KAT와 전체 하네스가 통과했고 `harness_fails=0`이었다. A/B 8-seed의 public key, secret key, ciphertext, encapsulated secret, warm/cold decapsulated secret, 훼손 ciphertext의 암묵적 거부 출력은 모두 바이트 단위 mismatch 0이었다. timed 셀의 마지막 출력과 stack canary도 mismatch/failure 0이었다. 같은 run 안의 동일 모드 drift는 최대 0.00683%, 독립 run의 같은 위치 효과 차이는 최대 0.00116%p로, 사전 등록한 0.3% 한계를 크게 밑돌았다. 두 플래시의 code-flash readback 853,584바이트는 같은 SHA-256 386A7ACD...BBCF5를 냈다.

7.4 배포형 절대 성능과 해석 경계

동일 ELF A/B에는 런타임 디스패처가 양쪽에 존재한다. 따라서 표 7의 절대 cycle은 상대효과 산출에만 쓰고, 최종형 절대값은 디스패처를 제거한 별도 빌드에서 측정했다(표 8).

cold는 warm의 1.804배이며 차이 626,867 cycle은 키생성 627,069 cycle과 0.03% 안에서 일치한다. 즉 cold 추가 비용은 사실상 `expandDecapsulationKey`가 수행하는 키생성 한 번이다. 측정 ELF의 SHA-256은 E9D32784...E334이고, 전체 소스, ELF/SREC/map, 두 원시 로그와 해시 manifest를 함께 동결했다. 이 결과는 한 EK-RA8M1 개체와 현재 코드·데이터 배치에 대한 cycle 결과다. 다른 Cortex-M85 구현, 에너지, Cortex-M55에 대해서는 미측정이다.

Table 8. 디스패처 없는 draft-10 최종형의 절대 성능.

연산	cycle 프로파일
키생성	627,069 packed 32-byte seed 입력
캡슐화	988,456 64-byte encapsulation seed
역캡슐화	780,020 warm, 확장키 캐시
역캡슐화	1,406,887 cold, packed 32-byte seed

8 상수 시간

스티칭은 구성상으로도 측정상으로도 타이밍 채널을 추가하지 않는다. 구조적으로: 이 변환은 컴파일 시점에만 일어난다—명령어를 재정렬하고, 레지스터를 이름 바꾸고, (-ffixed 컴파일을 통해) 고정된 스택/DTCM 오프셋으로의 스펬을 삽입한다. 분기는 하나도 넣지 않고 비밀 의존 주소 지정도 없다. 두 입력 스트림 모두 애초에 상수 시간이다(fiat-crypto의 생성 보장, Lenngren의 마스킹된 cswap 사다리, Keccak은 고정 순열, 4-way 큐 길이는 연산당 상수 43–45). M85에서 정수와 MVE 지연은 오퍼랜드 독립적이고 ITCM/DTCM은 0-wait 이므로, 고정된 명령어 시퀀스는 고정된 사이클을 함의한다. 경험적으로: 스티치된 fiat×MVE 커널을 서로 무관한 두 입력 집합에서 측정하면 중앙값 $\Delta = 0$ 사이클이고($\times 1000$ 루프 해상도에서 979,065 대 979,065), Unicorn 명령어 수도 모든 커널에서 두 입력에 대해 일치한다. dupect 스타일 통계는 돌리지 않았다. 이 구성에서 값 의존적 타이밍이 발생할 물리적 경로가 없으므로, 두 입력 점점은 1차 증거가 아니라 확인에 해당한다. 전력 부채널은 범위 밖이다.

9 관련 연구

Intel의 스티칭 [7]이 기원이다. 그것은 수작업 배치이고, 낭비 쪽을 측정하지 않았으며, 대칭 프리미티브 전용이고, 비순차 x86 전용이다. SLOTHY [1,2]는 단일 스트림을 최적화한다. 우리의 확장은 2-스트림 문제를—어떤 고정 분할 수작업 기법으로도 표현할 수 없는 스트림 간 레지스터 재할당까지 포함하여—솔버 문제로 만든다. Becker–Kannwischer의 스칼라/벡터 Keccak [4]은 Cortex-A에서 혼합 레지스터 파일 실행을 개척했고, 우리의 짝짓기 스펙트럼 실험은 그것이 순차 실행 M 계열에서 언제 통하는지를 정확히 정량화한다(로드는 짝을 이루고 스토어는 막는다). 멀티코어 하이브리드 병렬화 [12,6]는 직교적이다. 그쪽은 중첩을 실리콘으로 사고, 우리는 발행 슬롯에서 회수한다. M4에서의 하이브리드 비용 회계 [13]는 우리 기준선을 고정해 주었고 우리가 사전 등록한 스트림 균형 추정치를 제공했다. M85 측정은 그 방향을 확인해 주며, X25519가 더 긴 스트림이고 상한을 정하는 것은 슬롯 용량이 아니라 커버리지 회석임을 보인다. ML-KEM 구현 기준선: pqm4 [8], FIPS 203 [11].

10 결론

본 연구는 Cortex-M85 위의 draft-10 X-Wing에서 채택된 모든 최적화를 하나의 ELF로 통합하고, 기준형 대비 키생성 25.967%, 캡슐화 20.249%, warm 역캡슐화

11.224%, cold 역캡슐화 18.298%의 사이클 감소를 실측했다. 네 경로 모두 빨라졌으며 키생성과 캡슐화는 사전 등록한 20% 목표를 넘었다. 디스패처를 제거한 최종형은 각각 627,069, 988,456, 780,020, 1,406,887 cycle이다. 이 결론은 draft-10 공식 shared-secret vector, 8-seed A/B 전 출력, 암묵적 거부, 두 독립 플래시의 A-B-B-A 재현성으로 지지된다.

동시에 모든 그럴듯한 최적화가 살아남지는 않았다. 함수 스티칭은 국소 커널에서 자동화할 수 있었고 스칼라와 MVE의 완전한 동시 발행도 확인됐지만, 실제 상태 수명과 레지스터 재물질화를 보존한 X25519 제곱×NTT 병합은 순차 기준보다 느렸다. 더 이른 U 통합도 코드 배치 오류를 교정한 뒤 세 연산 모두 1.88–3.72% 회귀했다. 따라서 빈 발행 슬롯은 필요조건이지 충분조건이 아니다.

실무자를 위한 요약은 세 가지다. (i) 프로토콜 버전과 cold/warm API 프로파일을 먼저 고정하라. (ii) 개별 커널의 개선률을 더하지 말고, 모든 채택 변경을 같은 ELF와 같은 입력으로 다시 측정하라. (iii) MVE는 packing/CBD처럼 데이터 병렬성이 명확한 경로에 우선 적용하고, 교차 스케줄은 레지스터 수명과 저장 차단까지 포함한 실물 커널에서 판정하라.

한계. 누적률은 한 EK-RA8M1 개체와 현재 code/data placement에서의 same-ELF 상대효과다. 다른 M85 SoC, Cortex-M55/M52, 에너지, 전력·전자기 부채널은 미측정이다. 공식 test vector의 shared secret은 완전 대조했지만 public key와 ciphertext는 확보한 prefix만 외부 대조했다. 스티칭의 음성 결과는 본 연구가 사용한 필드 표현과 통합 방식에 한정하며, 다른 표현의 부호로 일반화하지 않는다.

References

1. Abdulrahman, A., Becker, H., Kannwischer, M.J., Klein, F.: Fast and clean: Auditable high-performance assembly via constraint solving (2022), iACR ePrint 2022/1303
2. Abdulrahman, A., Kannwischer, M.J., Lim, T.H.: Enabling microarchitectural agility: Taking ml-kem and ml-dsa from cortex-m4 to m7 with slothy. In: AsiaCCS (2025)
3. Barbosa, M., Connolly, D., Duarte, J.D., et al.: X-wing: The hybrid kem you’ve been looking for. IACR Communications in Cryptology **1**(1) (2024), ePrint 2024/039
4. Becker, H., Kannwischer, M.J.: Hybrid scalar/vector implementations of keccak and sphincs+ on aarch64 (2022), iACR ePrint 2022/1243
5. Connolly, D., Schwabe, P., Westerbaan, B.E.: X-wing: General-purpose hybrid post-quantum kem. Internet-Draft extttdraft-connolly-cfrg-xwing-kem-10 (2026), work in Progress, 2 March 2026, <https://datatracker.ietf.org/doc/draft-connolly-cfrg-xwing-kem/10/>
6. Eum, S.W., Song, M.H., Seo, H.J.: Performance analysis of a thread pool-based parallel execution model for hybrid post-quantum tls 1.3 handshakes (2026), iACR ePrint 2026/440
7. Gopal, V., Feghali, W., Guilford, J., et al.: Fast cryptographic computation on intel architecture processors via function stitching. Tech. Rep. 323686, Intel (2010)
8. Kannwischer, M.J., Petri, R., Rijneveld, J., Schwabe, P., Stoffelen, K.: pqm4: Testing and benchmarking NIST PQC on ARM Cortex-M4. <https://github.com/mupq/pqm4>, commit cc2c1b992b60

9. Langley, A., Hamburg, M., Turner, S.: Elliptic curves for security. Tech. Rep. RFC 7748, IETF (2016)
10. Lenngren, E.: X25519 for ARM Cortex-M4. <https://github.com/Emill/X25519-Cortex-M4> (2017)
11. National Institute of Standards and Technology: Module-lattice-based key-encapsulation mechanism standard. Tech. Rep. FIPS 203, NIST (2024)
12. Segatz, F., Al Hafiz, M.I.: Efficient implementation of crystals-kyber key encapsulation mechanism on esp32 (2025), arXiv:2503.10207
13. Sim, M., Lee, M., Cho, S., Hyoun, Y., Seo, H.: Evaluating hybrid kem/dsa for kpqc and nist pqc on arm cortex-m4 (2026), iACR ePrint 2026/1414

A 측정 세부사항

EK-RA8M1, 480 MHz, FSP BSP 클럭; DWT CYCCNT, $N=100$ 중앙값. 최종 중단간 값은 원시 cycle 중앙값이고, 25-cycle 하네스 보정은 짧은 microbenchmark에만 적용했다. 스티칭 microbenchmark는 코드 ITCM·데이터 DTCM에 고정했다. 최종 full X-Wing은 실제 배포 배치와 같은 hot ITCM section + code flash/I-cache 구성을 사용하고 스택과 주요 작업 버퍼를 DTCM에 두었다. 우리 보드 개체에서는 PMU 이벤트 카운터가 동작하지 않아, 프런트엔드/백엔드 스톱 귀속은 cycle과 별도 microbenchmark의 명령어 회계로 대체했다. Unicorn 명령어 수는 IT 블록을 개별 계산하므로 INST_RETIRED와 몇 퍼센트 차이가 날 수 있으며 최종 성능 분자에는 사용하지 않았다. 버전 고정: pqm4 cc2c1b992b60, 상류 SLOTHY 241093fa5ddc(+ 우리 패치 v0.1-v0.4), Lenngren X25519는 라이선스와 함께 벤더링. 하네스는 타이밍 전에 KAT와 reference 일치를 검사한다.

최종 누적 실험의 SHA-256 식별자는 ELF E9D32784, SREC AC0F88F이다 (전문은 동결 manifest에 수록). 측정 ELF는 .text=846,028 B, .data=7,572 B, .bss=393,740 B였다. 전체 소스, map/link 인자, harvester, 사전 등록 문서, 두 원시 로그와 SHA-256 manifest는 expCJ 동결 아티팩트에 보존했다.

B 재현 시 주의사항

루프 본문에는 .balign 16이 필요하다(없으면 반복당 ± 1 사이클). 큰 ITCM 함수는 리터럴 풀 대신 movw/movt를 써야 한다. DTCM 스택 전환 시 MSPLIM도 함께 옮겨야 한다. PMU가 비활성화된 상태에서는 CYCCNT 쓰기가 무시된다. 벤더 IDE가 TrustZone 플래시 파티션을 펌웨어 크기보다 작게 조용히 줄여 놓을 수 있다 (파티션 CLI로 수정). 컴파일러 출력을 솔버에 먹일 때는 더블워드 접근을 분할하고 스킵 슬롯을 절대 재사용하지 말아야 한다(절 4).